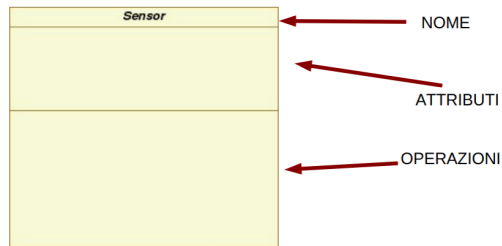


Lezione 3 09/10/2023 class diagram + information hiding + generalizzazione

martedì 10 ottobre 2023 12:50

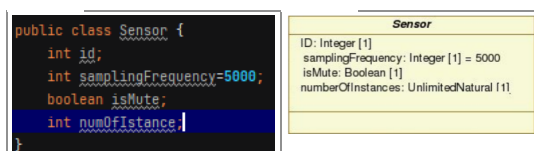
Facciamo un riepilogo sul class diagram : diagramma che rappresenta la struttura statica del sistema. Ha una logica grafo. Determina quindi i tipi di relazione (conoscenza tra classi) e determina il tipo di interazione tra loro . Attenzione a possibili classi "isolate" .(non vi è comunicazione con altre classi). Nel class diagram ci sono vari tipi di relazione : **generalizzazione , associazione , aggregazione , realizzazione** . Da notare che questo diagramma si usa con qualunque metodologia e qualunque livello di astrazione. Potrebbe quindi fare da pilota nella fase di implementazione ovvero si focalizza su aspetti statici del sistema (tipi di dato e relazioni) , quindi riassumendo descrive tutti i scenari possibili di interazione. Addentriamoci ora nel class diagram : ogni classe viene rappresentata così :



Dove il **nome** rappresenta una stringa , ovvero il nome di un'entità : in base al caso d'uso se java o uml , viene rappresentato secondo due convenzioni diverse : in java "package . nome_classe" in uml " prefisso :: nome_classe" . Vediamo un esempio :

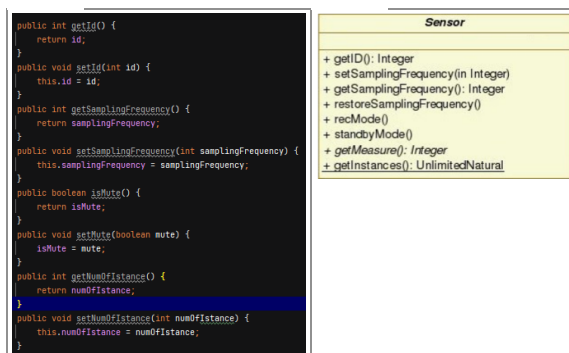
```
Public void abstract Sensor {  
/*  
    qui va il corpo della classe  
*/  
}
```

Vediamo ora il reparto **attributi** : rappresenta il nome ed i tipi di dato che la classe conosce : questi attributi sono condivisi da tutte le istanze di una stessa classe , ma con valori in generale diversi :



Da notare che nel caso della rappresentazione in java (classe) è la jvm che decide la rappresentazione dei vari tipi di dato. Riassumendo quindi : sia la rappresentazione java che quella in UML sono 2 linguaggi diversi : 2 possibili rappresentazioni di una rappresentazione di una conversione tra linguaggi . Non è possibile sempre effettuare una mappatura tra i due linguaggi. Quindi le scelte di interpretazioni possono essere delegate ad analista del sistema , al progettista od in base ad ambiente di sviluppo.

Vediamo ora il comparto **operazioni** : che cosa fa la classe , ovvero come viene modificato lo stato degli oggetti/istanze . Hanno una segnatura (tipo , nome , parametri) . Quindi si rappresentano le funzionalità delegate : funzionalità che la classe offre alle altre classi , se vi è relazione tra queste classi . Vediamo il dettaglio :



Attenzione : in java il tipo di ritorno non fa parte della segnatura del metodo .

```
public float add(int a, int b){}  
public int add(int a, int b){}
```

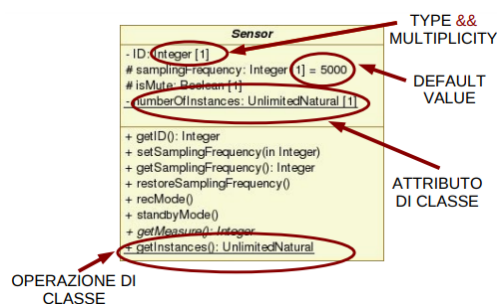
Vediamo ora tre particolari operazioni : **costruttori , distruttori e this** :

1. Costruttore : particolare operazione che ha la responsabilità di creare istanze. Concetto di basso livello. Usato nel mondo della programmazione oo. 1 costruttore ha ambito di classe (non necessariamente su istanza) . **Una classe può avere più costruttori !** I costruttori hanno stesso nome e ritornano lo stesso tipo . Rappresentano comportamenti implementativi . Attenzione che in uml non esistono proprio .

```
public Sensor(){}
public Sensor(int id){
    this.id=id;
}
a. public Sensor (int id, boolean booleanisMute)
{
    this.id=id;
    this.isMute=booleanisMute;
}
```

- Nel primo caso se presente ok, altrimenti il precompilatore inietta il costruttore di default vuoto.
 - Nell'ultimo invece i parametri formali servono per mitigare parte dell'istanziamento.
2. Distruttore : più complicata della creazione, perché deve garantire la consistenza dei dati/stato. In java la distruzione viene fatta implicitamente dal garbage collector (thread) , che dichiara i distruttori : questo thread controlla che per ogni oggetto vi siano collegamenti e /o puntatori che riferiscono ad istanze (ricordiamo che java non esiste algebra puntatori , quindi utente non può lavorare su heap).
 3. This : attributo speciale di ogni classe. **Indica il riferimento all'istanza corrente** : sia interno ai costruttori che ai metodi .

Torniamo al class diagram : focalizziamoci su attributo NumberOfInstances ed operazione getInstances : entrambe sono sottolineate :



Quindi si parla che questo attributo ha valore in **ambito di classe** : per quanto riguarda attributo il valore è **condiviso tra tutte le istanze**, inoltre non richiedono una istanza della classe per essere impiegati, mentre per quel che riguarda le operazioni non richiedono una istanza per essere invocate : questi "oggetti" vengono collegati agli invocanti a tempo di compilazione e non a quello di esecuzione.

```
package constructors;

public class Sensor {
    4 usages
    int id;
    2 usages
    int samplingFrequency=5000;
    3 usages
    boolean isMute;
    2 usages
    int numofInstance;

    2 usages
    static int attr1;
    public static int getAttr1() {
        return attr1;
    }

    public static void setAttr1(int attr1) {
        Sensor.attr1 = attr1;
    }
}
```

```
package constructors;

public class Prova {

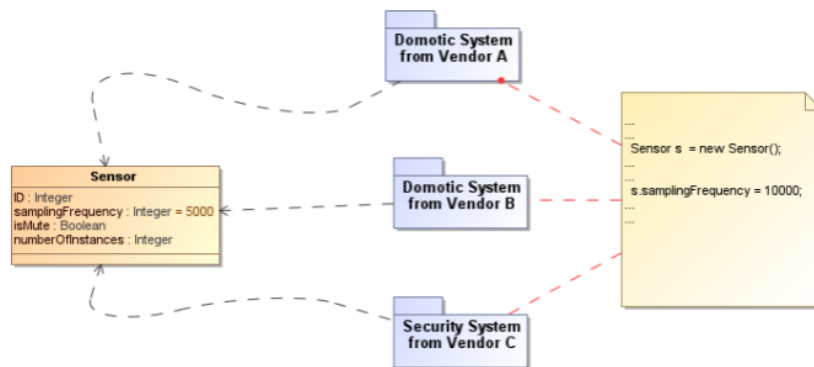
    int n1=Sensor.getAttr1();
    1 usage
    Sensor s=new Sensor();
    int n2=s.getNumofInstance();
}
```

Qui accedo a classe Sensor(o meglio attributi) da classe esterna

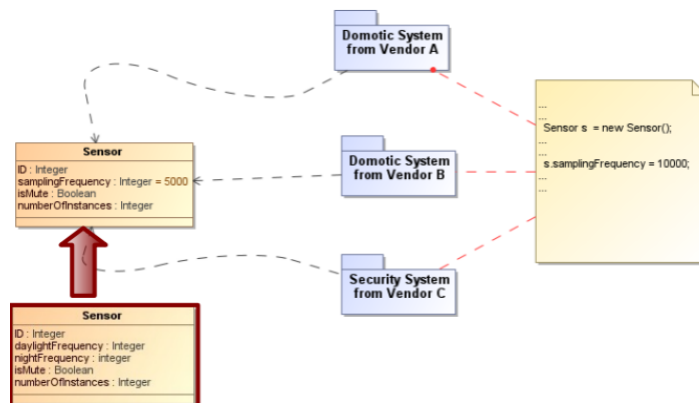
Andiamo a vedere ora invece come il concetto di static , viene implementato dalla jvm:



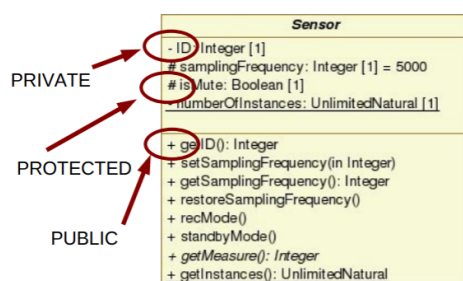
Vediamo ora un esempio pratico :



In questo caso da notare che il sistema non è efficiente, in quanto ogni sotto sistema usa e modifica il sensore a suo piacimento, portando così ad inconsistenza dei dati nel sistema. Quindi una parziale soluzione sarebbe definire un'altra versione di sensor, in modo da usare quella come scheletro :



Quindi la consistenza nello stato dell'oggetto ed impatto delle modifiche dello stato rispetto a quello base, suggeriscono di usare il principio di information hiding (incapsulamento) : separazione tra le specifiche di una funzionalità e la sua implementazione (separazione semantica del dato e come questa variabile viene implementata). **Quindi questo principio porta a nascondere le scelte che possono essere soggette a cambiamento.** La gestione delle situazioni non previste viene effettuata nel metodo opportuno. In dettaglio : questo principio usa **criteri di visibilità** :



1. Private : attributo/operazione riferita solo ed esclusivamente solo dalla classe che la dichiara.
2. Public : operazione/attributo riferita da tutti ed ovunque (possibili messaggi tra classi)
3. Protected : via di mezzo tra public e private. Un attributo/operazione così definita viene riferita nella stessa classe, nello stesso pacchetto ed in sottoclassi.

```

package sensor;
public class Sensor {
    1 usage
    private int id;
    2 usage
    protected int samplingFrequency=5000;
    3 usage
    protected boolean isMute;
    4 usage
    private static int numberOfInstances;

    public int getID(){
        return this.id;
    }
    public int getSamplingFrequency() {
        return samplingFrequency;
    }

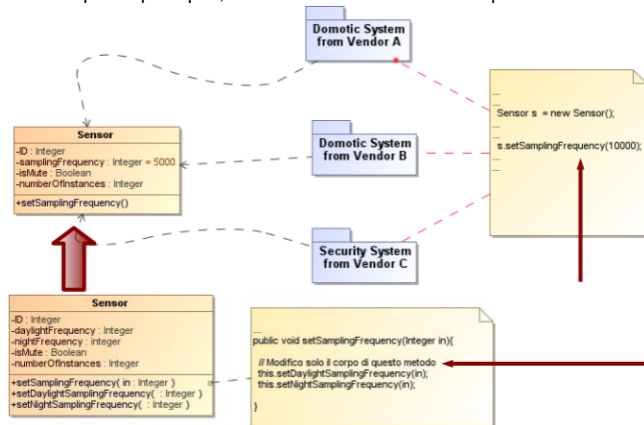
    public void setMute(boolean mute) {
        isMute = mute;
    }
    5 usage
    public void setSamplingFrequency(int samplingFrequency) {
        samplingFrequency = samplingFrequency;
    }
    public void restoreSamplingFrequency(){}
    public void recMode(){}
    public int standbyMode(){}
    public int getMeasure(){}
    return 300;
    }
    public static int getNumberOfInstance(){}
    return numberOfInstances;
    }
}

package sensor;
public class TestClass {
    public void testMethod()
    {
        Sensor s=new Sensor();
        s.setSamplingFrequency(23);
        /*
        s.id=45
        s.isMute=true
        these lines cannot be executed -> visibility
        */
    }
}
  
```

Da notare che private e protected potrebbero essere simili, ma non lo sono.

Quindi in generale : gli attributi sempre privati , operazioni che leggono/scrivono attributo sempre public : sono chiamati setter/getter.

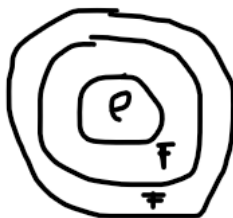
Quindi in virtù di questo principio , valutiamo soluzione finale del problema del sensore :



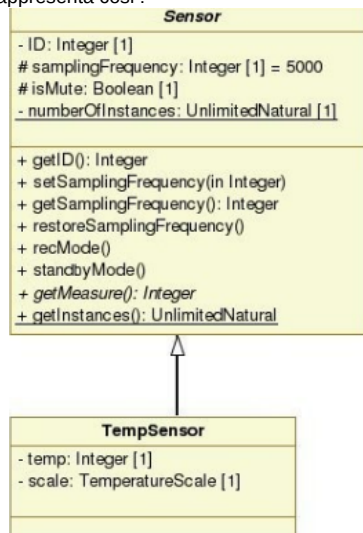
Tornando alle relazioni nel class diagram, andiamo a vedere quali sono :

PATH TYPE	NOTATION	REFERENCE
Aggregation		See "AggregationKind (from Kernel)" on page 38.
Association		See "Association (from Kernel)" on page 39.
Composition		See "AggregationKind (from Kernel)" on page 38.
Dependency		See "Dependency (from Dependencies)" on page 62.
Generalization		See "Generalization (from Kernel, PowerTypes)" on page 71.
InterfaceRealization		See "InterfaceRealization (from Interfaces)" on page 89.

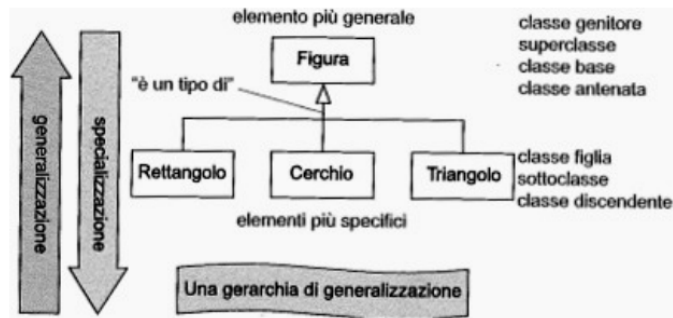
Concentriamoci ora su quella che è uno dei 4 cardini di java : la relazione di **generalizzazione** , la quale implica il concetto di **ereditarietà** : relazione tra una classe generica ed una più specifica . La classe più generica modella concetti più laschi, mentre la classe figlia esprime concetti più dettagliati . Relazione che induce una forte correlazione tra padre e figlio, in quanto la classe figlia eredita tutte le caratteristiche del padre una ed una sola volta . Questa relazione induce la relazione is-a-kind of : ovvero quello che è il padre lo è il figlio : la classe figlio ha più dettagli rispetto alla classe padre .



In uml una generalizzazione si rappresenta così :



- Nel nostro caso la sotto-classe (TempSensor) , oltre a ereditare le caratteristiche del padre, definisce proprietà ed attributi aggiuntivi, i quali sono solo suoi. Per avere ereditarietà si possono usare due processi equivalenti : nessuno prevale su altro : **generalizzazione vs specializzazione** :



Ma come buona pratica si cerca prima di individuare le classi più generici e poi quelli più specifici.

In java se si parla di generalizzazione o meglio di ereditarietà si usa la parola chiave **extends** :

```
package sensor;

public class Sensor {

    private int id;

    protected int SamplingFrequency=5000;

    protected boolean isMute;

    private static int numberOfInstance;

    public int getId(){return this.id;}

    public int getSamplingFrequency(){return SamplingFrequency;}

    public void setMute(boolean mute){this.isMute = mute;}

    public void setSamplingFrequency(int samplingFrequency){SamplingFrequency = samplingFrequency;}

    public void restoreSamplingFrequency() {}

    public void recMode() {}

    public void standByMode() {}

    public int getMeasure(){return 300;}

    public static int getNumberOfInstance(){return numberOfInstance;}

}

package sensor;

public class TempSensor extends Sensor{

    public String getAtt1() {

        return att1;

    }

    public void getAtt1(String att1) {

        this.att1 = att1;

    }

    public int getOtherAtt() {

        return otherAtt;

    }

    public void getOtherAtt(int otherAtt) {

        this.otherAtt = otherAtt;

    }

    2usages
    private String att1;
    2usages
    private int otherAtt;

    public TempSensor() {

    }

}
```

Per quanto riguarda invece i costruttori : in virtù che il costruttore riesce ad accedere a tutti i dati senza restrizione della classe a cui riferisce, mentre quella della classe figlia non riesce ad accedere alla parte privata dello stato della classe padre . Quindi nel momento in cui invoco il costruttore della classe figlia , il primo step è assicurarsi la costruzione corretta del costruttore della classe padre : si garantisce la configurazione corretta dello stato della classe padre , avendo così consistenza dei dati. Quanto detto finora può essere studiato ed applicato in un contesto multilivello :

